

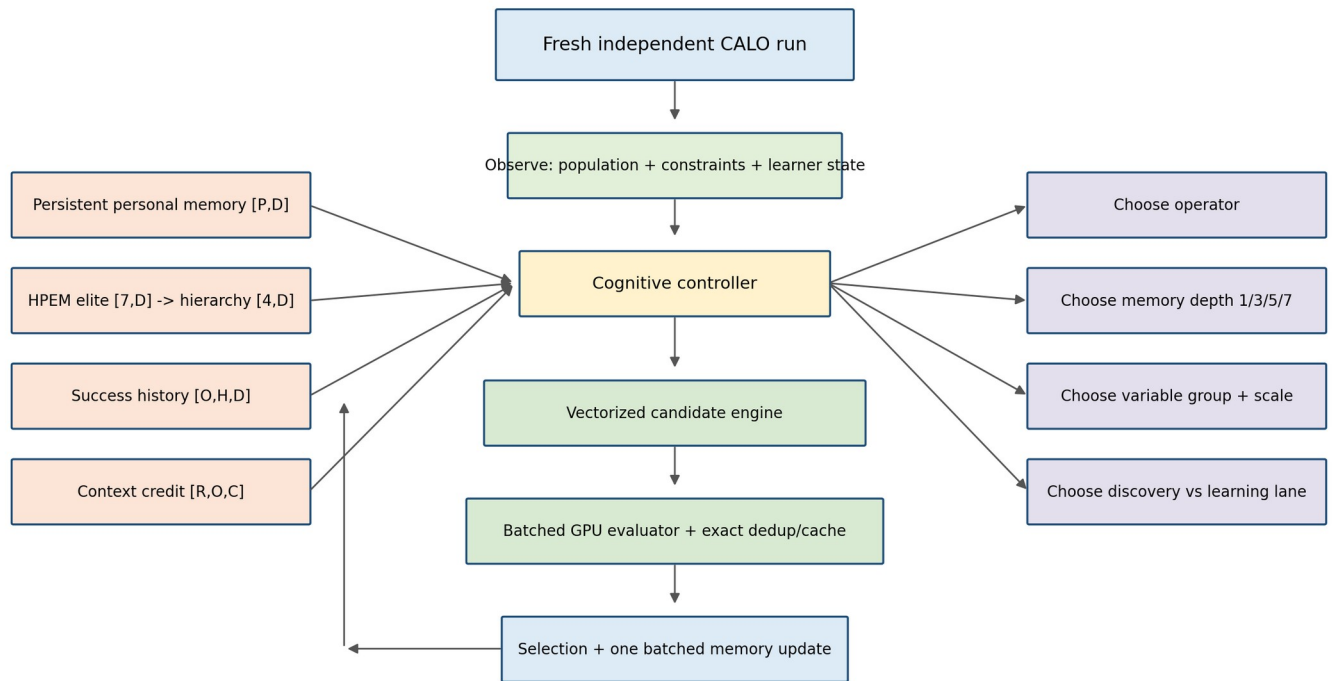
CALO vNext

Scientific Upgrade and Hybrid-Dimensional Tensor Architecture Plan

Maximum persistent dimensionality: 3D

Design objective

Transform CALO from a multi-operator adaptive optimizer into a coherent constraint-cognitive learning system that selects the right knowledge source, memory depth, operator, variable group, direction, and search scale for the current state - while preserving strict benchmark fairness, low memory, accelerator-first execution, and a fixed common function-evaluation budget.



CALO vNext: tensor-native closed-loop learning with a maximum permanent dimensionality of 3D

Planning basis: current CALO Core v2 observations, previously identified convergence bottlenecks, hierarchical Best-1/3/5/7 memory concept, and the agreed hybrid-dimensional tensorization strategy. This document proposes an implementation and validation roadmap; it does not claim literature novelty until a dedicated prior-art review is completed.

Contents

1. 1. Executive design decision
2. 2. Scientific targets and non-negotiable fairness rules
3. 3. Why CALO needs a structural upgrade
4. 4. Proposed CALO vNext closed-loop architecture
5. 5. Hybrid-dimensional tensorization strategy (1D, 2D, 3D only)
6. 6. Hierarchical Prefix Elite Memory (Best-1/3/5/7)
7. 7. Persistent individual memory and learner identity
8. 8. Contextual operator and memory learning
9. 9. Constraint cognition and adaptive epsilon
10. 10. Variable-group intelligence for mixed ORPD controls
11. 11. Dual-lane blind-discovery and learned-search design
12. 12. Cognitive precision engine for the lowest convergence floor
13. 13. Diversity, recovery, and anti-memory
14. 14. GPU-resident execution and low-memory engineering
15. 15. Proposed module and data structure
16. 16. CALO vNext algorithm flow and pseudocode
17. 17. Scientific innovation pillars and claim boundaries
18. 18. Ablation, validation, and benchmark plan
19. 19. Versioned implementation roadmap
20. 20. Final recommended structure

Primary success criterion

CALO should be judged by the lowest statistically reliable median final feasible objective under identical paired seeds and identical requested evaluation budgets, supported by convergence AUC, feasibility probability, variance, independent AC-power-flow validation, and appropriate statistical testing. A single lucky best run is not sufficient.

1. Executive design decision

The recommended upgrade is a tensor-native CALO vNext that keeps permanent state in the lowest natural dimensionality, uses temporary 3D tensors only where the third axis has real scientific meaning, and unifies memory, constraint cognition, operator learning, variable-group learning, recovery, and precision search into one closed feedback loop.

- Use 1D for scalar-per-candidate or scalar-per-category state.
- Use 2D for canonical population, personal memory, elite memory, and compact hierarchical summaries.
- Use 3D only for relationships such as operator x history x variable, regime x operator x context, candidate x memory-level x variable, or batched scenario/network chunks.
- Never duplicate Best-1, Best-3, Best-5, and Best-7 as four separate stores; derive them from one canonical seven-row elite tensor.
- Keep the scientific hot loop device-resident and batch all memory/credit updates once per evaluation batch.
- Use a single-budget blind-discovery plus learned-search strategy inside each independent CALO run; do not hide two full evaluation budgets inside one reported run.
- Concentrate the final portion of the budget on a CALO-native cognitive precision mechanism that learns productive variables, directions, and local search radii.

Recommended identity

CALO vNext should be positioned conceptually as a constraint-cognitive, hierarchical-memory, self-learning optimizer rather than as a collection of unrelated metaheuristic operators. The innovation is the closed-loop coordination among cognition, memory depth, context, variable type, operator choice, and precision/recovery behavior.

2. Scientific targets and non-negotiable fairness rules

Priority	Target	Engineering interpretation
1	Lowest convergence floor	Lowest median final FEASIBLE objective, not merely the lowest single run.
2	Fast computation	Maximize device-resident batched work; reduce Python loops, allocation churn, and CPU-GPU synchronization.
3	Low memory	Store canonical state once; use views, broadcasting, prefix statistics, chunking, and reusable scratch buffers.
4	Robust feasibility	Reach and retain feasibility without sacrificing the final objective.
5	Scientific reproducibility	Fresh optimizer state per repeated run, paired seeds, fixed FE budget, independent validation, frozen final algorithm.

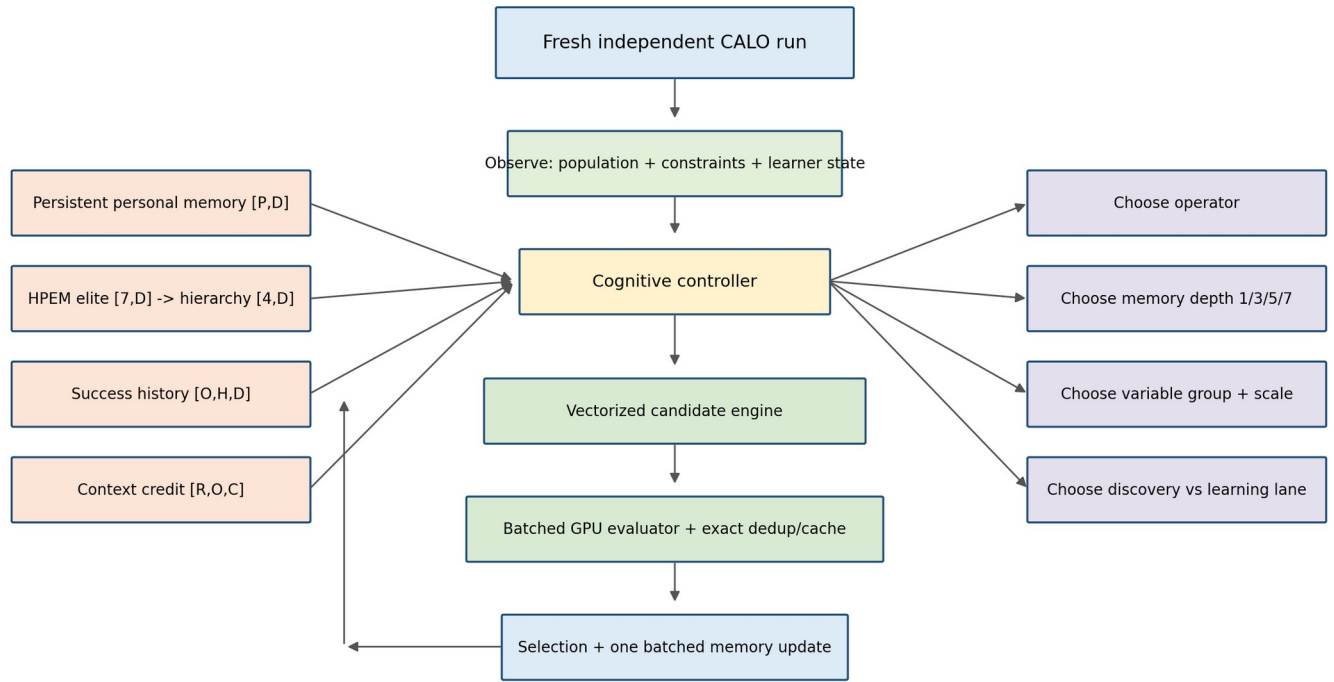
- CALO receives the same requested function-evaluation budget as every competitor.
- No previous statistical run may modify the next run during the final comparison.
- Any internal blind, learning, recovery, or precision phase must share the same total run budget.
- Exact evaluation cache hits may reduce physical solver calls, but a requested evaluation still consumes a budget unit for fairness.
- Historical learning, if studied, must be a separately declared transfer-learning variant or frozen before holdout comparison.
- IEEE 118 and IEEE 300 should be treated as holdout cases after development/freeze on smaller development cases.

3. Why CALO needs a structural upgrade

The present CALO concept already contains strong ingredients - constraint-aware state, multiple operators, feasible and boundary knowledge, online credit, success memory, mixed-variable awareness, and policy support. The main risk is that these mechanisms can dilute one another or fail to preserve useful learning continuity. The upgrade should first correct the learning foundations, then add only mechanisms that measurably lower the final objective.

Observed weakness	Why it can hurt convergence	vNext correction
Personal-best continuity can be lost	The personal direction collapses toward zero after state reset; learner-specific experience is not persistent.	Persistent learner identity and pbest inheritance across environmental selection.
Global operator credit	An operator successful during feasibility search may be overused during precision search.	Contextual credit indexed by regime/operator/context in a compact 3D tensor.
Order-dependent credit decay	Earlier successes can be discounted simply due to processing order.	Aggregate outcomes and update credit once per batch/operator.
Recovery parameter not fully operational	A learned action can be predicted without changing the actual population.	Use adaptive partial recovery fraction on weak/stagnated learners.
Elite collapse	Several "best" memories may represent the same basin.	Quality-diversity admission and Best-7 diversity role.
Weak final precision	Good early progress can stall above TLBO/DE-class final values.	Cognitive precision engine with sensitivity, successful direction, and adaptive radius.
Fixed/simple epsilon schedule	Constraint relaxation may tighten too early or too late.	Behavior-driven epsilon using feasibility and violation-progress signals.
AI/rule/credit dilution	Multiple blended controllers may cancel useful learned behavior.	Ablation-calibrated authority and measurable contribution of each control source.

4. Proposed CALO vNext closed-loop architecture

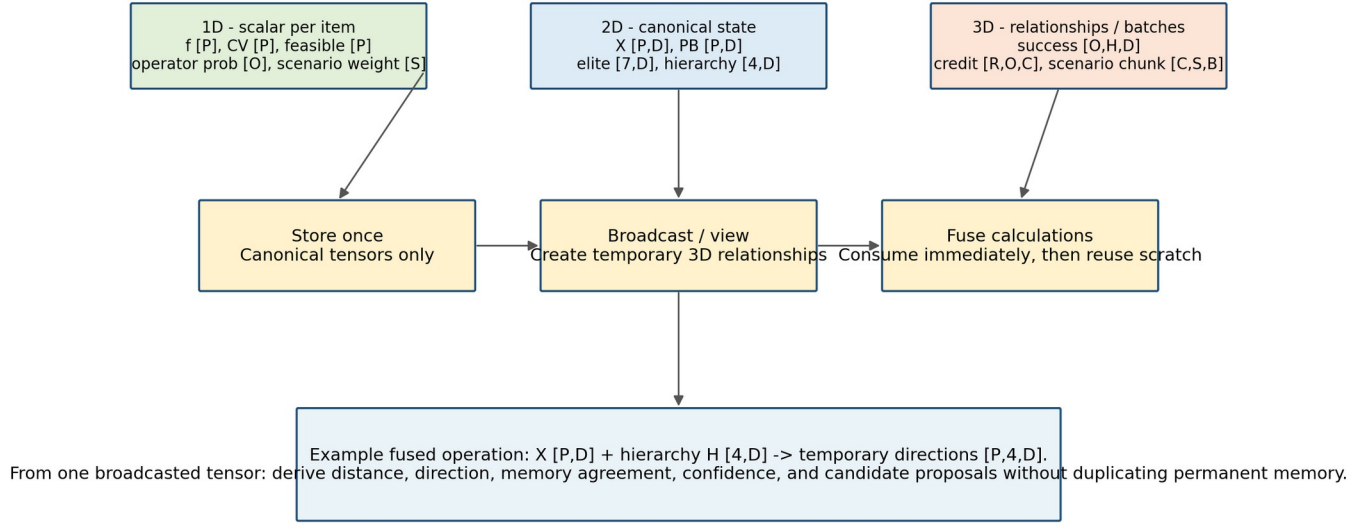


CALO vNext: tensor-native closed-loop learning with a maximum permanent dimensionality of 3D

The closed loop has five coordinated decisions: (1) what optimization condition is present, (2) which operator is appropriate, (3) which memory depth should be trusted, (4) which variable group and step scale should be modified, and (5) whether the current batch should favor discovery or learning. The outcome of each evaluated batch updates all learning signals together, eliminating per-candidate ordering artifacts.

Layer	Responsibilities	Must not do
Observe	Compute population, feasibility, diversity, stagnation, elite dispersion, variable-group, and budget signals.	Do not modify candidates.
Reason	Assign global and individual cognitive contexts; estimate memory confidence and lane balance.	Do not call the evaluator.
Learn	Update success history, contextual credit, HPEM summaries, sensitivity, and recovery evidence.	Do not duplicate canonical memory.
Act	Choose operator, memory depth, teacher, variable group, scale, and lane; generate vectorized candidates.	Do not exceed the common FE budget.
Evaluate	Run common ORPD evaluator in GPU-resident batches and return objective/constraint evidence.	Do not change scientific equations per algorithm.

5. Hybrid-dimensional tensorization strategy (1D, 2D, 3D only)



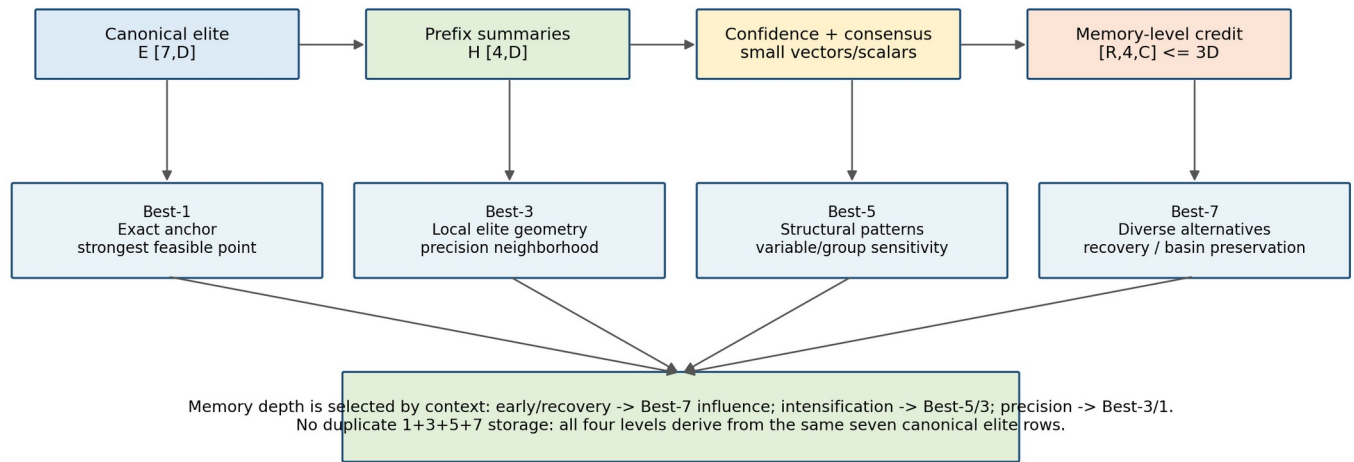
Rule: use the lowest natural dimension for storage; use 3D only where the third axis encodes real knowledge, context, history, or batch structure.

State / tensor	Shape	Dimension	Why this shape
Objective / CV / feasibility	[P]	1D	One scalar per learner; extra axes add no information.
Operator probability	[O]	1D	One probability per operator.
Population X	[P,D]	2D	Canonical learner x decision-variable state.
Persistent personal best PB	[P,D]	2D	One personal memory vector per learner.
Canonical elite E	[7,D]	2D	Single source of truth for Best-1/3/5/7.
Hierarchy H	[4,D]	2D	Compact derived summaries for the four memory depths.
Success history	[O,Hs,D]	3D	Which direction worked for which operator and history slot.
Contextual operator credit	[R,O,C]	3D	Which operator worked in which regime/context.
Group intelligence	[R,G,K]	3D	Regime x variable-group x compact statistics.
Memory directions (temporary)	[P,4,D]	3D	All learner-to-memory directions computed together, then released/reused.
Scenario/network scratch (chunked)	[C,S,B] or [C,S,L]	3D	Chunked candidate x scenario x bus/branch state; avoids global 4D storage.

Tensor law

Do not increase dimensionality for appearance. A tensor gains a new axis only when that axis represents a real relationship that is processed jointly. Memory is minimized by reducing stored elements, avoiding copies, reusing scratch, and chunking - not by reshaping 2D data into 3D.

6. Hierarchical Prefix Elite Memory (HPEM): Best-1 / Best-3 / Best-5 / Best-7



HPEM - Hierarchical Prefix Elite Memory

HPEM stores only seven canonical elite vectors. Best-1, Best-3, Best-5, and Best-7 are hierarchical interpretations of the same store, not four duplicated archives. Prefix-weighted summaries and compact confidence signals are recomputed once per batch.

Memory level	Scientific role	Typical use
Best-1	Exact elite anchor	Final preservation, strongest destination, fine precision center.
Best-3	Local elite geometry	Local center/spread, memory consensus, precision neighborhood.
Best-5	Structural elite intelligence	Variable/group sensitivity and recurring patterns separating better elites.
Best-7	Diverse high-quality alternatives	Alternative basins, recovery teachers, diversity guardian, anti-collapse behavior.

Recommended canonical slot policy: protect slot 0 as the absolute best feasible solution; use slots 1-2 for additional objective elites; use slots 3-4 for quality-diversity structural elites; use slots 5-6 for strong but meaningfully diverse alternatives. Admission must follow the common constraint/feasibility logic and reject near-duplicate decoded controls.

Canonical store: E [7,D]
 Prefix mask: M [4,7]
 Derived hierarchy: H = prefix_weighted_summary(M, E) -> [4,D]
 Temporary directions: Delta = H[None,:,:] - X[:,None,:] -> [P,4,D]
 Consume Delta to obtain distance, direction, confidence, consensus and proposals; then reuse/release

scratch.

7. Persistent individual memory and learner identity

Each learner must preserve a persistent personal knowledge path rather than resetting pbest to the current survivor after every environmental selection. This gives CALO a stable individual-learning channel analogous in purpose (not mechanism) to trajectory memory in established optimizers.

- Each survivor retains its associated personal-best vector and personal-best evaluation.
- An accepted offspring inherits the lineage identity of the parent it replaces, then updates pbest only if the offspring dominates the stored personal best under the common feasibility logic.
- A newly restarted learner receives a new lineage ID and initializes personal memory from its restart point.
- Personal memory should remain [P,D] plus compact 1D metadata; no 3D history of every past personal best is required.
- A small success-direction tensor stores only useful compressed transitions, not full trajectories.

Expected effect

Persistent personal memory restores learner continuity and gives success-direction learning a non-zero, individualized reference. This is a P0 correction because it can materially affect exploitation quality before any new sophisticated mechanism is added.

8. Contextual operator and memory learning

Replace a single global operator score with compact context-dependent learning. The maximum persistent dimensionality remains 3D.

Tensor	Shape	Meaning
operator_credit	[R,O,C]	Credit for operator O under cognitive regime R and compact context C.
memory_credit	[R,4,C]	Credit for Best-1/3/5/7 depth under the same context.
success_history	[O,Hs,D]	Compressed successful directions organized by operator and history slot.
group_stats	[R,G,K]	Attempts, success, objective gain, feasibility gain, effective step scale by variable group.

The context index should remain compact, for example: feasible/high-diversity, feasible/low-diversity, infeasible-improving, infeasible-stagnated. Regime and context should not explode into dozens of categorical combinations; the design should maximize learning density per stored parameter.

Update rule: accumulate batch-level attempts, successes, normalized objective gain, normalized constraint gain, and feasibility transitions; then perform one exponentially smoothed update per operator/context after the batch. This eliminates candidate-order bias and supports GPU-friendly aggregation.

9. Constraint cognition and adaptive epsilon

CALO should treat constrained optimization as a sequence of distinct cognitive problems rather than a single scalar penalty search. The constraint controller observes decomposed violation, feasibility ratio, epsilon-feasible ratio, boundary-memory quality, violation improvement velocity, and stagnation.

Condition	Cognitive interpretation	Response
Low feasibility, violation improving	Promising route toward feasibility	Maintain or gently tighten epsilon; prioritize boundary learning.

Low feasibility, violation stagnated	Constraint deadlock	Trigger repair/recovery, increase boundary/diversity influence.
High feasibility, stable progress	Feasible region established	Tighten epsilon faster and shift toward objective learning.
Stable feasibility, late stage	Constraint search largely solved	Set epsilon to zero; spend budget on feasible precision.
Feasibility lost during precision	Over-aggressive local move	Reduce trust radius and increase Best-3/constraint-safe guidance.

Key principle

A feasibility-producing operator must not automatically dominate final objective refinement. Feasibility reward and objective reward are different learning signals and should be normalized and credited separately before contextual combination.

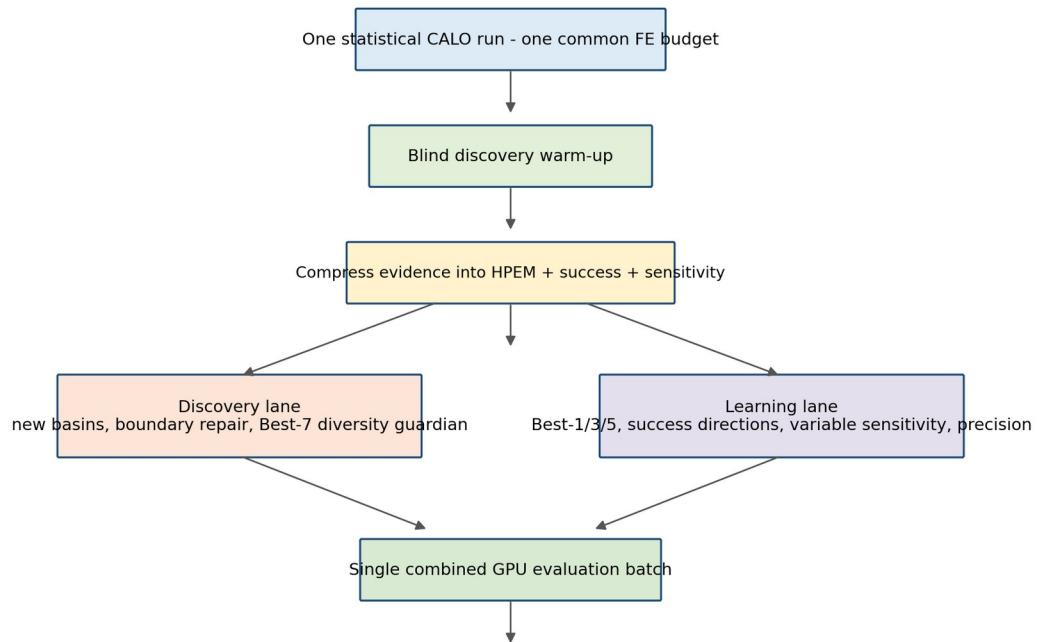
10. Variable-group intelligence for mixed ORPD controls

CALO should explicitly learn at the control-group level because generator voltage, transformer taps, and shunt compensation have different smoothness, sensitivity, and discrete structure.

Group	Representation	Learning action
Generator voltage	Continuous	Directional/sensitivity learning, trust-radius refinement, weighted elite geometry.
Transformer taps	Discrete stepped	State-transition success statistics; neighborhood moves only on legal tap values.
Shunt compensation	Discrete stepped	Context-specific step transition statistics; avoid invalid continuous averaging.
Optional future groups	Problem-specific	Add only when the decoder exposes a coherent variable family.

The compact tensor `group_stats[R,G,K]` can store attempts, success probability, normalized objective gain, normalized constraint gain, and useful step magnitude. Candidate generation then allocates more probability to groups that still produce meaningful improvement, especially in the precision phase.

11. Dual-lane blind-discovery and learned-search design



Fairness: blind + learning + precision are internal phases; their requested evaluations together must never exceed the same run budget given to competitors.

A full blind run followed by a second full learning run would be scientifically unfair and roughly double expensive power-flow work. The recommended design is one independent statistical run with an internal blind-discovery warm-up and a dynamically increasing learning lane, all under the same total requested evaluation budget.

Internal phase	Purpose	Budget behavior
Blind discovery warm-up	Build feasibility routes, diversity, initial HPEM, success directions, operator evidence, and variable sensitivity.	Consumes an early portion of the same run budget.
Mixed dual-lane search	Discovery lane protects alternative basins while learning lane uses HPEM/context knowledge.	Both lanes are combined into the same GPU evaluation batch.
Cognitive precision	Concentrates remaining evaluations on high-value feasible refinement while retaining a small diversity guardian.	Still within the original total budget; no hidden rerun.

The transition should eventually be behavior-driven using a Memory Readiness Score based on feasible ratio, elite stability, diversity, successful-history density, and operator evidence. An easy case can activate learning early; a difficult case can remain discovery-heavy longer.

Statistical independence

Run N may learn from its own earlier blind-discovery evaluations, but it must not import adaptive runtime memory from Run N-1 during the primary comparison. Every repeated run starts from a fresh optimizer state and its assigned seed.

12. Cognitive precision engine for the lowest convergence floor

The precision engine is the primary mechanism intended to lower the final convergence floor after feasibility is stable. It should be CALO-native, evaluation-counted, mixed-variable aware, and activated by behavior rather than merely by a fixed iteration percentage.

- Use Best-1 as the exact anchor and Best-3 as the local elite geometry.
- Use Best-5 to infer which variables or variable groups still distinguish stronger from weaker elites.
- Use recent successful directions from success_history[O,Hs,D] as a directional prior.
- Use an adaptive trust radius: maintain/expand after reliable improvement; shrink after repeated failure.
- For discrete variables, search legal neighbor states using learned transition success instead of continuous averaging.
- Accept only genuine improvements under the common feasibility/dominance rule; every trial consumes an evaluation request.
- If memory consensus becomes low or the local radius collapses without progress, hand control temporarily back to Best-7/diversity recovery.

Conceptual continuous proposal:

$$x_{\text{trial}} = x_{\text{elite}} + r_{\text{t}} * d_{\text{success}} + \beta_{\text{t}} * d_{\text{memory}} + \sigma_{\text{t}} * \text{noise}$$

where r_{t} is a learned/adaptive trust radius, d_{success} is a normalized successful-direction prior, d_{memory} comes from Best-1/3/5 hierarchy, and all terms are bounded/decoded by the common ORPD decoder.

Why this matters

Many algorithms reach a good region but lose the comparison in the final small improvements. CALO vNext should explicitly reserve enough counted evaluations for high-quality precision rather than continuing broad exploration until the budget is exhausted.

13. Diversity, recovery, and anti-memory

Recovery should preserve learned knowledge while replacing only unproductive learners. The learned recovery_fraction must become an actual control variable, bounded by safe minimum/maximum fractions and triggered only by evidence of stagnation/diversity collapse.

- Protect the best feasible solution, strong personal memories, HPEM, and useful success directions.
- Replace only the worst/stagnated fraction; initialize replacements from a mixture of global exploration, Best-7 diverse regions, and under-explored variable groups.
- Maintain a compact anti-memory of repeatedly harmful operator/context or variable-transition patterns rather than storing entire failed populations.
- Use recovery as a temporary mode; once useful improvement resumes, reduce its authority automatically.

Anti-memory can be stored as compact failure statistics within the same contextual credit and group-stat tensors, avoiding another large archive. Its purpose is not to forbid exploration permanently but to reduce repeated waste in the same context.

14. GPU-resident execution and low-memory engineering

Engineering rule	Implementation
Device-resident scientific state	Keep X, PB, E, H, success history, credit, and group statistics on CUDA/XPU when the common evaluator supports it.
Single batched update	Update HPEM, success memory, credit, sensitivity, and regime statistics after the evaluated batch - not per candidate.
Reusable scratch pool	Preallocate candidate, ranking, scenario, bus/branch, and direction scratch buffers and reuse them.
Broadcast instead of repeat	Use views/expand/broadcasting where possible; avoid materializing duplicated Best-1/3/5/7 tensors.
Chunk scenario/network state	Use temporary 3D chunks [C,S,B] / [C,S,L] rather than persistent huge tensors.
Exact deduplication/cache	Canonicalize decoded controls, solve unique candidates once, scatter results

	back; still count every requested evaluation.
Asynchronous telemetry	Transfer only small downsampled summaries to the GUI; never stream entire populations every iteration.
FP64 where scientific	Keep AC PF, objective, constraints, and optimizer decision values at required precision; compress IDs/flags/metadata separately.

The largest speed gains are expected from reducing repeated data movement and solver duplication, not from compressing the tiny HPEM tensors. The power-flow evaluator remains dominant, so unique-candidate batching, scenario chunking, persistent device constants, and minimized synchronization should receive priority.

15. Proposed module and data structure

```

calo/
  optimizer.py           # orchestration + common FE budget
  tensor_state.py        # CALOTensorState canonical device tensors
  cognitive_controller.py # global + individual context/regime decisions
  dual_lane_controller.py # blind/discovery vs learned-search allocation
  hierarchical_memory.py  # HPEM E[7,D] and H[4,D]
  success_memory.py       # finite success history [0,H,D]
  contextual_credit.py     # [R,O,C] and [R,4,C]
  variable_intelligence.py # [R,G,K], sensitivity and discrete transitions
  precision_engine.py     # CALO-native final refinement
  recovery_engine.py      # adaptive partial recovery without forgetting
  evaluation_cache.py     # exact canonical decoded-control dedup/cache
  operators/             # vectorized operator kernels

accelerated/
  calo_tensor_kernels.py
  scratch_pool.py
  population_batcher.py
  deduplication.py
  device_state.py

```

Data family	Canonical storage	Update frequency
Core population	X [P,D], f [P], CV [P], feasible [P]	Every evaluated batch
Personal memory	PB [P,D] + compact 1D metadata	After survivor/offspring selection
Elite memory	E [7,D], H [4,D]	Once per batch
Success memory	[O,Hs,D] + small scalar metadata	Once per batch/operator
Context credit	[R,O,C], [R,4,C]	Once per batch/context
Variable intelligence	[R,G,K] / compact sensitivity	Once per batch/group
Telemetry	Downsampled 1D time series	Throttled, asynchronous
Scratch	Preallocated temporary 2D/3D buffers	Reused, not historized

16. CALO vNext algorithm flow and pseudocode

```

Initialize fresh run from seed
Initialize X[P,D], PB[P,D], constraints, HPEM E[7,D]
Initialize success[0,H,D], contextual credits, group statistics
Set requested_FE = 0

while requested_FE < FE_budget:
  observe population + constraint + diversity + stagnation signals

```

```

compute global context and learner contexts
update memory readiness and discovery/learning lane ratio
derive H[4,D] from canonical E[7,D] using prefix summaries
estimate memory confidence + consensus
choose operator, memory depth, variable group and scale per learner
generate candidates vectorially (temporary 3D views only where useful)
canonicalize / deduplicate exact decoded controls
evaluate unique candidates in accelerator batches
count ALL requested candidates against FE budget
scatter cached/evaluated results back to requests
perform environmental selection and persistent pbest update
aggregate successes and failures for the whole batch
update HPEM, success memory, contextual credit, sensitivity once per batch
adapt epsilon, trust radius, lane ratio and recovery fraction
if stable feasible stagnation: increase cognitive precision authority
if low diversity + stagnation: partial recovery without forgetting

```

Return best independently valid feasible solution + complete telemetry

The design intentionally separates the requested evaluation budget from physical solver calls. Exact deduplication can reduce physical work, but cannot give CALO more optimization requests than a competitor.

17. Scientific innovation pillars and claim boundaries

Pillar	What makes the integrated CALO concept distinctive
Constraint cognition	The algorithm reasons from decomposed constraint state and feasibility progress, not only a scalar penalty or iteration counter.
Hierarchical memory depth	A single canonical elite tensor is interpreted at Best-1/3/5/7 resolutions, and CALO learns which depth is useful in the current state.
Contextual action selection	Operator, memory depth, variable group, direction, scale, and discovery/learning balance are selected jointly from current evidence.
Dual-lane self-learning	Each independent run first generates its own evidence and then exploits that evidence within the same fixed FE budget, while retaining a diversity guardian.
Cognitive precision	Late-stage search learns productive variables, directions, and trust radius instead of merely reducing a generic random step size.
Recovery without forgetting	CALO replaces only unproductive learners while retaining elite, personal, and success knowledge.
Tensor-native efficiency	Hybrid 1D/2D/3D storage fuses related calculations without permanent duplication, supporting GPU-resident execution and low memory.

Novelty guardrail

Do not claim that archives, success history, adaptive operators, reinforcement learning, population reduction, or trust-region ideas are individually novel. A defensible novelty claim must concern the exact integrated mechanism and must be confirmed by a formal literature/prior-art search before publication.

18. Ablation, validation, and benchmark plan

The full architecture must not be merged in one step. Use staged ablation so that every added mechanism earns its complexity by measurable improvement.

Stage	Variant	Acceptance evidence
-------	---------	---------------------

A0	Current CALO baseline	Freeze baseline seeds, FE budgets, telemetry, runtime, memory, feasibility and final objective.
A1	Tensor-state refactor only	Seeded numerical equivalence; lower allocations/transfers; no scientific regression.
A2	Persistent pbest + stable batched credit + active recovery	Better or equal median objective/feasibility with no extra FE.
A3	HPEM Best-1/3/5/7	Lower AUC/final objective; diversity not degraded.
A4	Contextual operator/memory credit	Higher useful success rate; lower wasted evaluations.
A5	Variable-group intelligence + adaptive epsilon	Faster feasibility and better mixed-variable refinement.
A6	Dual-lane self-learning	Lower median final objective under the exact same total FE.
A7	Cognitive precision	Material reduction in late-stage convergence floor.
A8	Final combined CALO vNext	Strongest minimal coherent set only; reject mechanisms with no robust benefit.

18.1 Development and holdout protocol

- Develop and tune architecture primarily on IEEE 30 and IEEE 57 with paired seeds.
- Freeze equations, mechanisms, hyperparameters, tensor policies, and learning logic before holdout.
- Evaluate frozen CALO unchanged on IEEE 118 and IEEE 300.
- For final evidence use at least 30 independent runs; support 50 for publication-grade statistics.
- Compare median/mean/best/worst/std/IQR feasible objective, feasible-run probability, convergence AUC, and evaluations-to-first-feasibility.
- Use paired statistical tests and multiple-comparison correction where appropriate; publication claims must exclude infeasible, failed, non-finite, or independently unverified evidence.

18.2 Performance instrumentation

- GPU memory peak and allocated/reserved memory.
- CPU memory and number of CPU-GPU transfers/synchronizations.
- Unique physical solver calls versus requested FE (cache/dedup efficiency).
- Candidates/second and power-flow solves/second.
- Fraction of time in evaluator, candidate generation, memory learning, GUI/telemetry, and synchronization.
- Operator/context success, memory-depth success, variable-group success, trust-radius behavior, and recovery effectiveness.

19. Versioned implementation roadmap

Milestone	Core deliverable	Gate
vNext-0	Instrumentation + reproducible baseline	No algorithm change; full seeded baseline captured.
vNext-1	CALOTensorState + scratch pool + device residency	Equivalent scientific results; lower overhead.
vNext-2	Persistent pbest, finite success memory, batch credit, operational recovery	Foundational learning correctness tests pass.
vNext-3	HPEM Best-1/3/5/7 + quality-diversity admission	Ablation shows robust convergence benefit.
vNext-4	Contextual operator/memory credit + group intelligence + adaptive epsilon	Reduced wasted evaluations and stronger feasibility/intensification transition.
vNext-5	Dual-lane self-learning under one FE budget	Outperforms single-lane CALO on development cases without extra FE.

vNext-6	Cognitive precision engine	Demonstrable late-stage objective improvement.
vNext-7	Freeze + IEEE 118/300 holdout + full portfolio	No post-holdout tuning.
vNext-8	Publication-grade 30-50 run evidence	Independent validation and statistical evidence complete.

20. Final recommended CALO vNext structure

The recommended final structure is summarized below. It deliberately limits persistent tensors to a maximum of 3D while allowing temporary broadcasted 3D computation where it fuses multiple calculations.

```

FRESH INDEPENDENT RUN
  |
  v
BLIND DISCOVERY WARM-UP
  | builds initial feasibility routes + evidence
  v
HPEM: E[7,D] -> H[4,D] (Best-1/3/5/7)
PERSISTENT PB[P,D]
SUCCESS[0,H,D]
CONTEXT CREDIT[R,0,C] + MEMORY CREDIT[R,4,C]
GROUP INTELLIGENCE[R,G,K]
  |
  v
COGNITIVE CONTROLLER
  chooses context + operator + memory depth + variable group + scale
  |
  v
DUAL-LANE SEARCH
  discovery guardian <-> learned search
  |
  v
SINGLE COMBINED GPU BATCH
  exact canonical dedup/cache + common ORPD evaluator
  |
  v
ONE BATCHED LEARNING UPDATE
  |
  v
COGNITIVE PRECISION / RECOVERY AS NEEDED
  |
  v
BEST INDEPENDENTLY VALID FEASIBLE RESULT

```

Recommended research thesis

CALO vNext should learn not merely "the best solution" or "the best operator", but which knowledge resolution, operator, variable group, direction, and search scale are appropriate for the current constrained optimization state. The hierarchical memory and dual-lane self-learning system should be evaluated as an integrated cognitive mechanism, with the final objective always obtained under the same requested evaluation budget as competitors.

20.1 Immediate implementation priorities

21. Instrument and freeze a reproducible baseline before modifying CALO behavior.
22. Fix persistent learner memory, numerical reward safety, batch-level credit, and operational recovery first.
23. Refactor canonical state into hybrid 1D/2D/3D tensors without changing scientific behavior.
24. Implement HPEM Best-1/3/5/7 from one E[7,D] source of truth.

25. Add contextual operator/memory credit and variable-group learning as compact 3D tensors.
26. Implement dual-lane self-learning within one common FE budget, not two full hidden runs.
27. Implement the cognitive precision engine and verify that it lowers the median final feasible objective.
28. Freeze on development cases, validate on holdout cases, then run full 30-50 run publication comparison.

End of planning document.